



BSc *Hons* in Information Technology

IT3091 : Machine Learning

Year 3 • Semester 1 • 2026

Faculty of Computing

LAB SHEET 05

Logistic Regression & k-Nearest Neighbors

Learning Outcomes:

By the end of this lab session, students will be able to:

- Explain the basic mechanics of Logistic Regression and k-Nearest Neighbors kNN .
- Implement the Sigmoid function, Log-Loss and Gradient Descent from scratch.
- Use a trained Logistic Regression model to produce probabilities and class predictions.
- Implement kNN from scratch using distance calculation and majority voting.
- Investigate the effect of different values of k kNN .
- Compare from-scratch implementations with Scikit-learn implementations.

Part A

Logistic Regression

A.1

Dataset & Problem Setup

A university wants to predict whether a student will **Pass** 1 or **Fail** 0 an examination based on the number of hours studied.

Hours Studied X	Result Y
1	0
2	0
3	1

Hours Studied X	Result Y
4	1
5	1
6	1

Task 1

1. Create NumPy arrays for X and Y .
2. Display the dataset using a scatter plot.
3. Identify the input feature, target variable.

Input feature - Hours studied X

Target Variable - Result Y

A.2

Sigmoid Function

Logistic Regression first calculates a linear score:

$$z = \theta_0 + \theta_1 x$$

The Sigmoid function converts the score into a probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Task 1

1. Implement the Sigmoid function from scratch using NumPy.
2. Initialize $\theta_0 = 0$ and $\theta_1 = 0$.
3. Calculate the predicted probabilities for all observations.
4. What probability is produced for each observation at the initial parameter values?
5. Why is the Sigmoid function required in Logistic Regression?

4. Every Observation has a probability of $p = 0.5$. This happens initially: $\theta_1 = 0$, $\theta_2 = 0$. Therefore, $z = 0$ and $\sigma(0) = 0.5$.

5. The linear equation can produce any real value from negative infinity to positive infinity. However, classification requires a probability between 0 and 1. The Sigmoid function maps

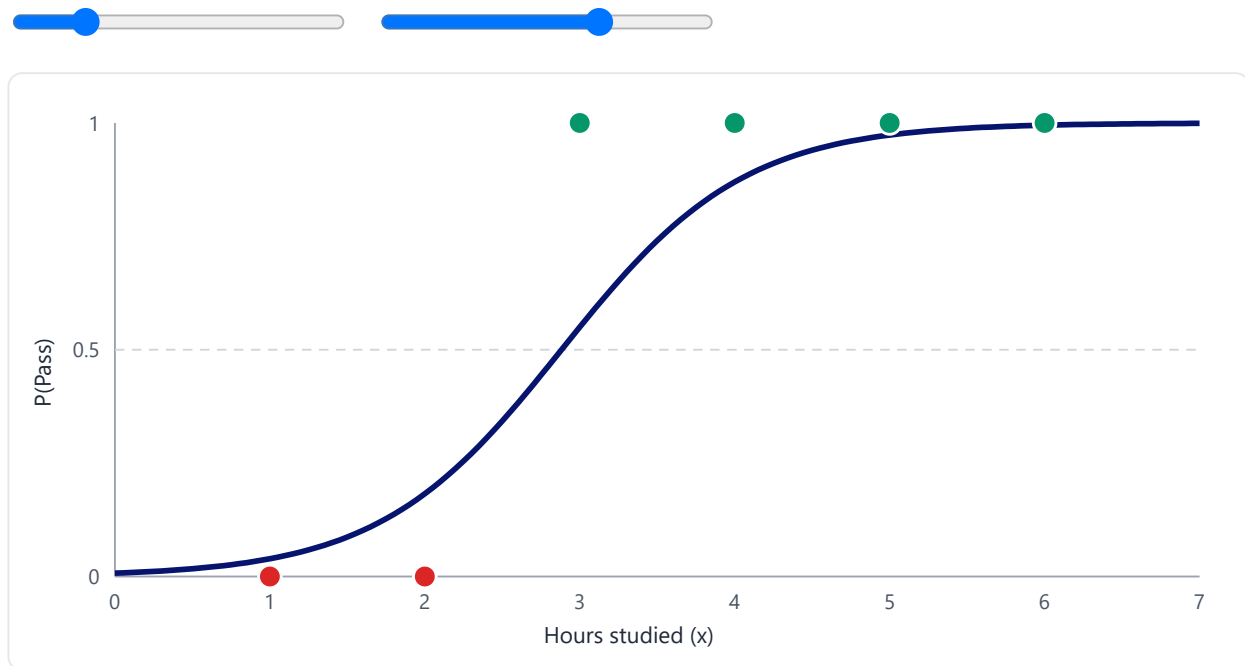
the linear score to the probability range. Therefore, Logistic Regression can interpret the result as the probability of belonging to class 1, which, in this problem, means "Pass".

Sigmoid Explorer

Move parameters and observe how the Logistic Regression probability curve changes. Notice that the output always remains between 0 and 1.

θ_0 : -4.9

θ_1 : 1.7



For $x = 2.5$ hours: $P(\text{Pass}) = \mathbf{0.343}$. With threshold 0.5, class = **Fail (0)**.

A.3

Log-Loss / Binary Cross-Entropy

Use the Binary Cross-Entropy cost function:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(h_i) + (1 - y_i) \log(1 - h_i)]$$

where:

$$y_i = Y$$

$$h_i = \text{Sigmoid}(z)$$

Task 1

1. Implement the Log-Loss function from scratch.
2. Calculate and display the initial cost.
3. What does a smaller Log-Loss value indicate?
4. What happens to the loss when the model confidently makes an incorrect prediction?
5. Why is a cost function required during model training?

3. A smaller Log-Loss means that the predicted probabilities are closer to the correct class labels. Therefore, the model is making better predictions. A value closer to zero generally indicates better performance.

4. The Log-Loss becomes very large. For example, if the actual class is 1 but the model predicts a probability close to theta, the model is confidently wrong. Log-Loss heavily penalizes this type of mistake.

5. A cost function measures how inaccurate the model's predictions are. Gradient Descent uses the cost to determine how the parameters should be adjusted. During training, the objective is to minimize the cost. Without a cost function, the model would have no mathematical measure telling it whether its predictions are improving.

A.4

Gradient Descent

Gradients:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m (h_i - y_i)$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^m (h_i - y_i) x_i$$

Update rule:

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

Task 1 – Train the Model

1. Set a suitable learning rate α and initialize θ_0 and θ_1 to zero.
2. Calculate the gradients.
3. Update the parameters using Gradient Descent.

4. Repeat the process for a sufficient number of iterations.
5. Store the cost at each iteration.
6. Display the final values of θ_0 and θ_1 .

```
Final theta0: -9.731632850221068
Final theta1: 3.9753167619356784
Final cost: 0.04444551763864804
```

Task 2 – Observe Convergence

1. Plot iteration number against Log-Loss.
2. What happens to the loss as training progresses?
3. What does it mean when the loss begins to stabilize?
4. What could happen if the learning rate is too large?
5. What could happen if the learning rate is too small?

2. The Log-Loss gradually decreases as Gradient Descent updates the parameters. This shows that the predictions are becoming closer to the actual target values.

3. When the loss starts to stabilize, the parameter updates are producing very small improvements. This indicates that Gradient Descent is approaching a minimum of the cost function and the model is converging.

4. If the learning rate is too large, Gradient Descent may take excessively large steps. It can overshoot the minimum and cause the loss to oscillate or even increase instead of converging.

5. If the learning rate is too low, Gradient Descent will still move toward the minimum, but very slowly. Therefore, many more iterations will be required for the model to converge.

Gradient Descent Visualization

Run Gradient Descent one step at a time and watch the fitted probability curve change while Log-Loss decreases.

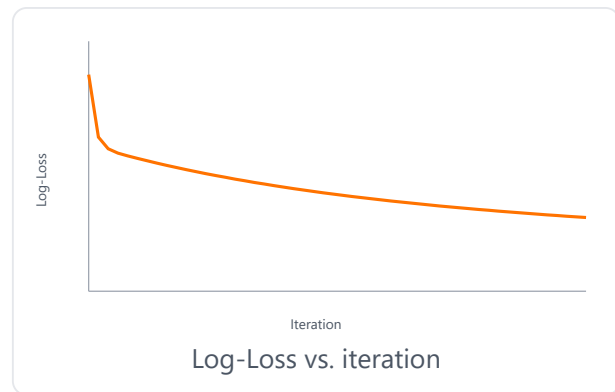
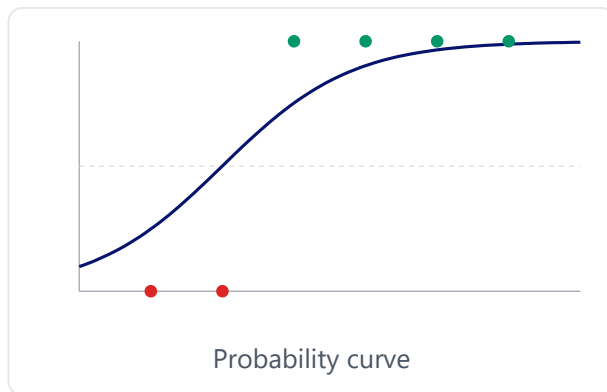
Learning rate α : 0.51



Step +1

Run 50

Reset



Iteration **51** | $\theta_0 = -2.214$ | $\theta_1 = 1.109$ | Loss = **0.2359**

A.5

Making Logistic Regression Predictions

For a new observation:

$$P(Y = 1|X) = \sigma(\theta_0 + \theta_1 X)$$

Using a threshold of 0.5:

$$\hat{y} = \begin{cases} 1, & P(Y = 1|X) \geq 0.5 \\ 0, & P(Y = 1|X) < 0.5 \end{cases}$$

Task

1. Create a prediction function from scratch.
2. Predict the result for a student who studies for **2.5 hours**.
3. Display the predicted probability and predicted class.
4. Change the threshold from 0.5 to 0.7. Does the prediction change?
5. Explain the purpose of a classification threshold.

3. Probability: 0.5514816708244734
Predicted class: 1

4. Probability: 0.5514816708244734
Predicted class: 0

At threshold 0.5: $0.55 \geq 0.5$ so pass, at threshold $0.55 < 0.7$ so fail

5. The threshold converts the probability produced by Logistic Regression into a class label. For example, with a threshold of 0.5, probabilities of 0.5 or higher are classified as Pass. Changing the threshold makes the classifier more or less strict when assigning the positive class.

A.6

Verify Logistic Regression Using Scikit-learn

1. Train `LogisticRegression` using the same dataset.
2. Obtain the predicted probability and class for 2.5 hours.
3. Compare the Scikit-learn result with your from-scratch implementation.
4. Are the learned parameter values exactly the same? Discuss possible reasons for differences.

4. The from-scratch implementation and Scikit-learn implementation both predict **Pass** for 2.5 hours when using a threshold of 0.5. However, their exact probability values and learned parameters may differ. Scikit-learn uses its own optimized numerical solver and applies regularization by default, whereas our implementation uses simple Gradient Descent with a manually selected learning rate and number of iterations. Therefore, the parameter values do not have to be identical. The lab explicitly asks you to discuss these possible differences.

Part B

k-Nearest Neighbors kNN

B.1

Problem Setup

Predict whether a student will **Pass** 1 or **Fail** 0 using two features: Hours Studied and Practice Tests Taken.

Student	Hours	Practice Tests	Result
A	1	1	0
B	2	1.5	0
C	2	3	0
D	4	5	1
E	5	4.5	1
F	5	2.5	1

New student: $\mathbf{Q} = 4, 2$.

B.2

Calculate Euclidean Distance

$$d(p, q) = \sqrt{\sum_{j=1}^n (p_j - q_j)^2}$$

1. Create the training dataset using NumPy.
2. Implement Euclidean distance from scratch.
3. Calculate the distance from Q to every training observation.
4. Display the distances and sort the observations from nearest to farthest.

```
F Distance = 1.118 Class = 1
B Distance = 2.062 Class = 0
C Distance = 2.236 Class = 0
E Distance = 2.693 Class = 1
D Distance = 3.0 Class = 1
A Distance = 3.162 Class = 0
```

B.3

Find the k Nearest Neighbors & Majority Vote

For $k = 3$, select the three smallest distances and predict using:

$$\hat{y} = \text{mode}\{y_{(1)}, y_{(2)}, \dots, y_{(k)}\}$$

1. Set $k = 3$.
2. Identify the three nearest observations.
3. Display their distances and class labels.
4. Apply majority voting.
5. Predict whether student Q will Pass or Fail.

```
Three nearest neighbors:
F Distance: 1.118 Class: 1
B Distance: 2.062 Class: 0
C Distance: 2.236 Class: 0
```

Student Q is predicted to Fail.

B.4

Investigate the Effect of k

1. Repeat the prediction for **k = 1, 3 and 5**.
2. Record the predicted class for each value of k.

k	Predicted Class
1	Pass(1)
3	Fail(0)
5	Pass(1)

1. Does the prediction change when k changes?
2. What happens when k is very small?
3. What happens when k is very large?
4. Why is an odd value of k commonly preferred for binary classification?

1. Yes. The prediction changes as k changes. For this dataset, k=1 predicts Pass, k=3 predicts Fail and k=5 predicts Pass.

2. A very small k makes the model highly sensitive to individual nearby observations. For example, with k=1, only F determines Q's class. This can produce high variance and make the classifier sensitive to noise or outliers.

3. With a very large k, many distant observations participate in the decision. This makes the model smoother and less sensitive to local structure, but it can cause underfitting because the majority class begins to dominate.

4. For binary classification, an odd value of k reduces the possibility of a tied vote. For example, with k=3, one class must receive at least two votes.

B.5

Visualize kNN

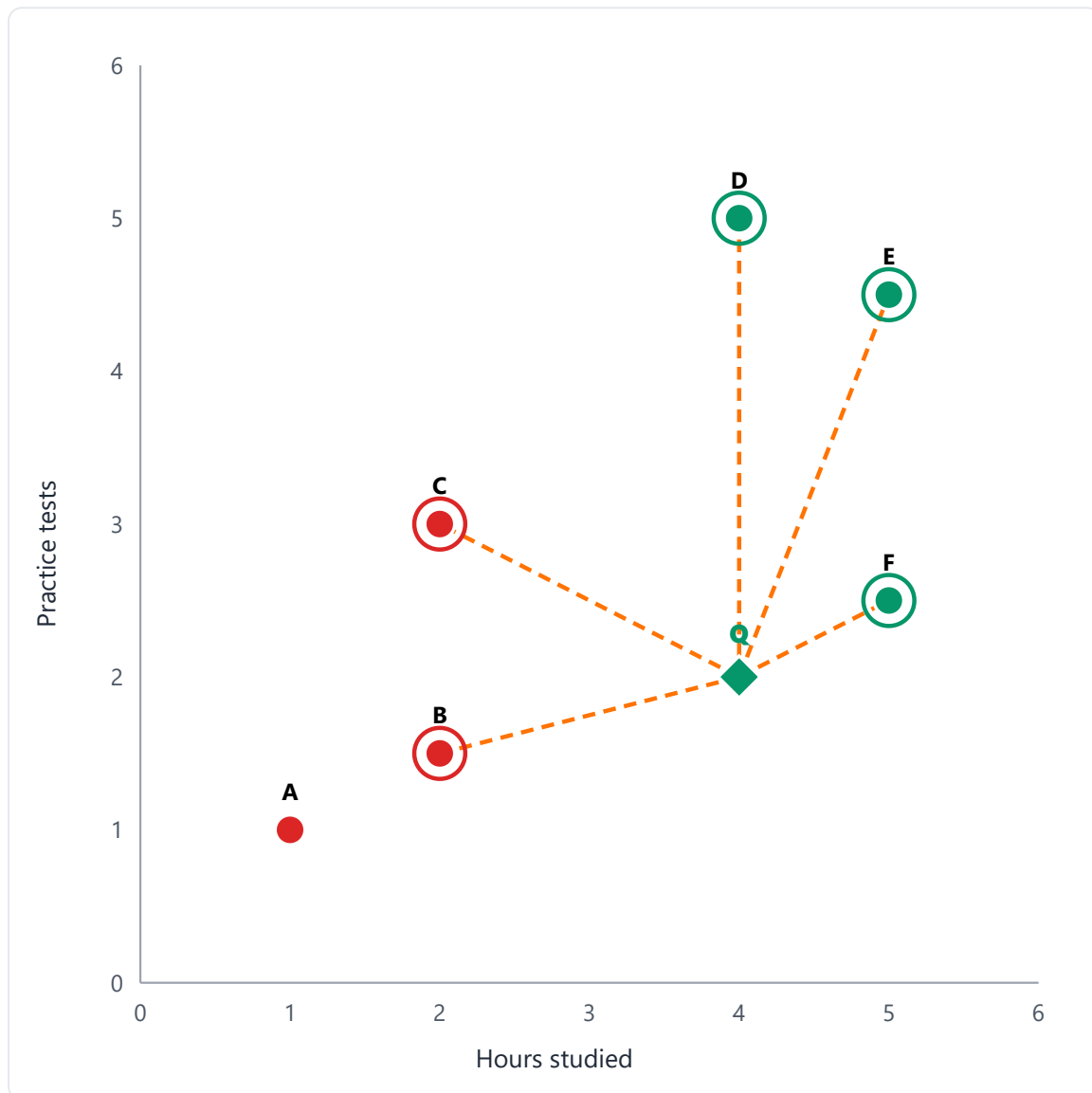
1. Create a scatter plot with Hours Studied on the X-axis and Practice Tests on the Y-axis.

2. Use different markers or colours for Pass and Fail observations.
3. Plot the query point Q on the same graph.
4. Identify the nearest neighbors to Q for $k=3$.

kNN Explorer

Click anywhere inside the graph to move the new student Q. Change k and observe which neighbors vote and how the prediction changes.

Choose k: ☐ 1 ☐ 3 ☐ 5



Q = (4.0, 2.0) | k = 5 | Nearest: F (1.12), B (2.06), C (2.24), E (2.69), D (3.00) | Prediction: **Pass**
(1)

B.6

Create the Complete kNN Algorithm

Combine the previous steps into a function:

```
knn_predict(X_train, y_train, query, k)
```

The function should:

1. Calculate distances.
2. Sort the distances.
3. Select the k nearest observations.
4. Obtain their labels.
5. Perform majority voting.
6. Return the predicted class.

Prediction: 0

Q = Fail

B.7

Verify kNN Using Scikit-learn

1. Train `KNeighborsClassifier` using the same dataset.
2. Set `n_neighbors=3`.
3. Predict the class of `Q = 4, 2`.
4. Compare the prediction with your from-scratch implementation.

Prediction: 0

The Scikit-learn KNN classifier predicts **Fail (0)** for `Q = (4,2)`. This is the same prediction produced by our from-scratch implementation. Both methods identify the three closest neighbors and use majority voting.

B.8

Why Feature Scaling Matters for kNN

Consider two features:

- Age: 18–70
- Annual Income: 20,000–500,000

1. Which feature would dominate Euclidean distance?
2. Why could this produce misleading nearest neighbors?
3. Which preprocessing technique can reduce this problem?
4. Standardize the kNN input features and repeat the prediction.

1. **Annual Income** would dominate. Its numerical range is much larger:

- Age: 18–70
- Annual Income: 20,000–500,000

2. Euclidean distance measures numerical differences directly. Therefore, a small difference in income can contribute much more to the distance calculation than a large age difference. As a result, KNN may effectively ignore age even if age is important for prediction.

3. This particular dataset: scaling does not change the final prediction: Fail

Part C

Understanding the Concepts

Answer each question analytically in 3-6 sentences: justify your reasoning rather than stating a definition, and where relevant, ground your answer in what you actually observed in the Sigmoid Explorer, Gradient Descent Visualization, or kNN Explorer above.

1. If Mean Squared Error were used instead of Log-Loss to train this Logistic Regression model, explain, by reasoning about the shape of each cost surface, why Gradient Descent would struggle to reliably reach the minimum.

Using Mean Squared Error together with the Sigmoid function can create a non-convex cost surface containing flatter regions and possible local minima. This makes Gradient Descent less predictable because it can move slowly in flat regions or fail to approach the global minimum reliably. Log-Loss, on the other hand, gives Logistic Regression a well-behaved convex cost surface. In the Gradient Descent visualization, the Log-Loss decreased steadily as the parameters were updated, illustrating why it is suitable for optimization. Therefore, Log-Loss gives Gradient Descent a more reliable path toward the optimal parameters.

2. A spam filter and a cancer-screening classifier both output a probability. Argue, in terms of the relative cost of false positives versus false negatives, whether each should use a threshold above, below, or exactly at 0.5.

A spam filter may use a threshold **above 0.5** if incorrectly marking an important legitimate email as spam is considered more costly than allowing some spam into the inbox. A higher threshold means the classifier requires stronger evidence before labeling an email as spam. In cancer screening, the threshold should generally be **below 0.5**, because a false negative could mean failing to identify a patient who actually has cancer. Lowering the threshold increases sensitivity and makes the system more likely to flag possible cases for further testing. Therefore, the appropriate threshold depends on the relative consequences of false positives and false negatives.

3. Using what you observed in the kNN Explorer, explain how increasing k moves the model along the bias-variance trade-off, and justify what the decision boundary would look like if k equaled the total number of training points.

When k is small, such as $k=1$, kNN is highly sensitive to individual training examples, resulting in **low bias but high variance**. Increasing k averages the decision across more neighboring observations, making the prediction smoother and reducing variance, but increasing bias. This behaviour can be observed in the kNN Explorer because different values of k can change Q 's predicted class. If k equals the total number of training observations, every query point would be classified according to the majority class of the entire dataset. The resulting decision boundary would therefore become extremely simple and would largely ignore local clusters.

4. A classmate claims: "kNN is strictly better than Logistic Regression because with $k=1$ it can always achieve 100% accuracy on the training set." Critically evaluate this claim, considering both training and unseen data.

The claim is incorrect because high training accuracy does not guarantee good performance on unseen data. With $k=1$, kNN can effectively memorize the training observations, giving it very low training error. However, this makes the model highly sensitive to noise and unusual observations and can result in overfitting. A slightly larger k may generalize better even if its training accuracy is lower. Logistic Regression may also outperform kNN when the true relationship between features and classes is approximately linear, so neither algorithm is strictly better in every situation.

5. Logistic Regression produced a straight decision boundary in this lab, while kNN's boundary could bend around clusters of points. Explain the mathematical reason for this difference by referring to each model's hypothesis function.

Logistic Regression models probability using:

$$P(Y=1|X) = \sigma(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots)$$

At a threshold of 0.5, the decision boundary occurs when the linear expression equals zero, producing a straight line in a two-dimensional feature space. kNN does not learn a

fixed linear equation; instead, it determines the class of each query from the labels of nearby training observations. Because different neighborhoods can contain different class majorities, the kNN decision boundary can bend and form irregular shapes around clusters. Therefore, Logistic Regression has a parametric linear boundary, while kNN creates a flexible, data-dependent boundary.



FACULTY OF COMPUTING